Testing With Minitest

If all that seems a little abstract, let's look at an example of how you use the minitest library to write automated testing. We'll start with a Roman numeral class. So far, the code is pretty simple: it just lets us create an object representing a certain number and display that object in Roman numerals:

unittesting/romanbug.rb

```
# This code has bugs
class Roman
  MAX_ROMAN = 4999

  def initialize(value)
    if value <= 0 || value > MAX_ROMAN
      fail "Roman values must be > 0 and <= #{MAX_ROMAN}"
    end
    @value = value
  end

  FACTORS = [
    ["m", 1000], ["cm", 900], ["d", 500], ["cd", 400],
    ["c", 100], ["xc", 90], ["l", 50], ["xl", 40],
    ["x", 10], ["ix", 9], ["v", 5], ["iv", 4],
    ["i", 1]
  ]

  def to_s
    value = @value
    roman = ""
    FACTORS.each do |code, factor|
      count, value = value.divmod(factor)
      roman << code unless count.zero?
    end
    roman
  end
end
```

We could test this without a framework code by writing another plain Ruby script, like this:

unittesting/manual_romanbug.rb

```
require_relative "romanbug"

r = Roman.new(1)
fail "'i' expected" unless r.to_s == "i"

r = Roman.new(9)
fail "'ix' expected" unless r.to_s == "ix"
```

However, as the number of tests in a project grows, this kind of ad hoc approach can get complicated to manage. The Ruby standard library comes with minitest, a framework originally written by Ryan Davis and the seattle.rb user group, that makes tests easier to write, run, and manage.

The minitest testing framework is three facilities wrapped into a neat package:

- It gives you a way of expressing individual tests.
- It provides a framework for structuring the tests.